



Une entreprise ordinaire à l'agilité extraordinaire

FORMATION JAVA SPRING ANGULAR

ALGO & CO

mohamed.el-babili@fms-ea.com

33+ 628 111 476

Version : 3.0

DMAJ : 18/12/23



Module : DEV-ALGO-001

Introduction à l'Algorithmique

01 Définir ce qu'est un algorithme et l'algorithmique

02 Développer le raisonnement informatique

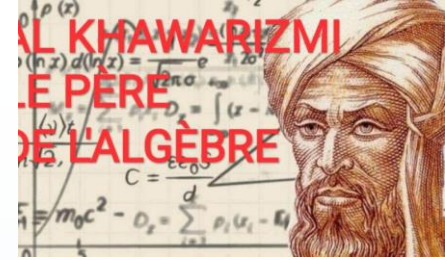
03 Analyser et réfléchir avant de concevoir et coder un programme

Sommaire



- Introduction
- Algorithme & Adn
- Plan ou recette d'un algorithme
- Un programme, c'est du gâteau !
- Variables et Entrées/sorties
- Les variables varient dans le temps
- Bien choisir ses variables ou structures de données
- C'est quoi une boucle
- Programme complet avec automatisation du process
- Algorithmique : méthode pour réaliser des algos (1) (2)
- Cas général & cas particuliers
- Outil visuel d'aide à la réalisation d'algo
- Déboguer : comment résoudre les imprévus ?
- QQ critères d'une application de qualité
- Répéter pour maîtriser
- Comment optimiser un algorithme + qualité du code ?
- Prise de sens ou déclic
- Paradigme de programmation
- C'est quoi le métier de développeur ?
- Pour un cerveau en bonne santé
- Ressources

Introduction



- Le premier à avoir systématisé des algorithmes est le mathématicien perse **Al-Khwârizmî**, actif entre 813 et 833. Dans son ouvrage Abrégé du calcul par la restauration et la comparaison, il étudie toutes les équations du second degré et en donne la résolution par des algorithmes généraux.
- **L'algorithmique** est l'étude et la production de règles et techniques qui sont impliquées dans la définition et la conception d'Algorithmes.
- Un **algorithme** est une suite finie clair d'opérations ou d'instructions permettant de répondre à un ou plusieurs besoins dans un langage informatique.

Algorithmme & Adn

5

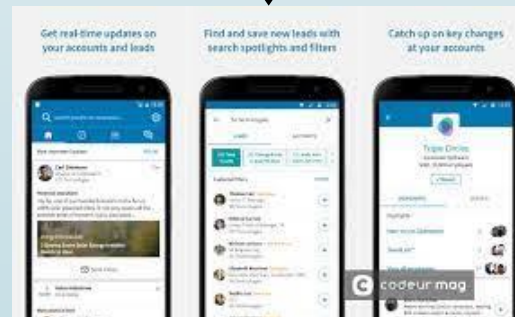
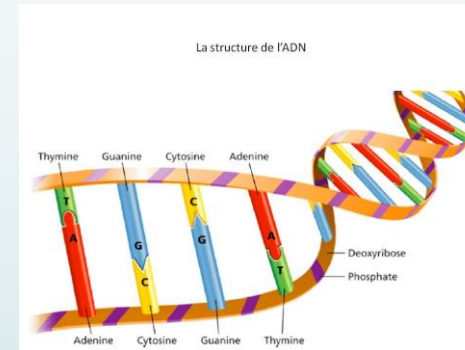
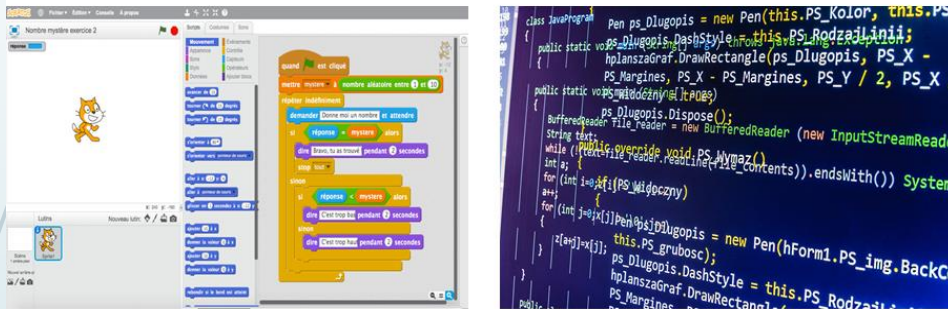
Variables

Entrées/Sorties

Tests

Boucles

Adenina (A)
Timina (T)
Citocina (C)
Guanina (G)



Plan ou recette d'un algorithme

- Pour réaliser un algorithme, il faut un plan ou recette qui répond d'abord à un besoin, ici on pourrait exprimer **la volonté de manger un gâteau**
- La première étape va consister à réunir les **ingrédients** dont nous aurons besoin selon des quantités précises
- Ensuite il faut suivre la recette **étape par étape** afin d'arriver au résultat final et à la dégustation



Ingrédients :

- 400 g de yaourt nature sucré ou yaourt aromatisé
- 4 œufs
- 40 g de Maïzena tamisée

Mélanger le yaourt avec les œufs

Ajouter la Maïzena tamisée

Humidifier un moule à gâteau

Froisser une feuille de papier sulfurisé et la mettre au fond du moule

Verser la préparation

Enfourner à 170 °C pendant environ 35 à 50 min selon les fours

Mettre au frigo 1 à 2 h

Un Algorithme est une suite d'instructions compréhensibles, qui une fois exécutée correctement, conduit à un résultat donné : un gâteau !

Un programme, c'est du gâteau

Variables

Entrées/Sorties

7

RecetteGateauVersion1.java x

```
1 package fr.fms;
2
3 public class RecetteGateauVersion1 {
4     public static void main(String[] args) throws InterruptedException {
5         /*-----Programme de réalisation d'un gâteau-----*/
6         System.out.println("Bienvenu dans notre programme qui réalise un gâteau au yaourt simple et gourmand");
7
8         /*-----liste des ingrédients-----*/
9         int yaourt = 400;
10        int oeuf = 4;
11        int maizena = 40;
12        System.out.println("quantité de yaourt = " + yaourt + "g");
13        System.out.println("nombre d'oeufs = " + oeuf);
14        System.out.println("quantité de maizena tamisée = " + maizena + "g");
15
16        System.out.println("mélanger les yaourts avec les oeufs pendant 10 secondes");
17        Thread.sleep(10000); //moyen pour donner l'impression qu'on mélange pendant 10 secondes
18
19        System.out.println("humidifier un moule à gâteau");
20        Thread.sleep(5000); //opération prend 5 secondes
21
22        System.out.println("Froisser une feuille de papier sulfurisé et la mettre au fond du moule");
23        Thread.sleep(5000); //opération prend 5 secondes
24        System.out.println("Verser la préparation");
25
26        System.out.println("Enfourner à 170° pendant 35 à 50 minutes selon les fours");
27        four(170, 35);
28
29        System.out.println("Enfin, mettre au frigo pendant 1 à 2h.");
30        frigo(60);
31
32        System.out.println("Régalez vous !");
33    }
34
35    public static void four(int temperature, int minutes) {
36        System.out.println("----->>>c'est parti pour la cuisson à " + temperature + "° et durant " + minutes + " minutes");
37    }
38
39    public static void frigo(int minutes) {
40        System.out.println("----->>>c'est parti pour le frigo pendant " + minutes + " minutes");
41    }
42 }
43
```

Console x

```
<terminated> RecetteGateauVersion1 [Java Application] C:\Users\EI-BabiliM\p2\pool\plugins\org.eclipse.just
Bienvenu dans notre programme qui réalise un gâteau au yaourt simple et gourmand
quantité de yaourt = 400g
nombre d'oeufs = 4
quantité de maizena tamisée = 40g
mélanger les yaourts avec les oeufs pendant 10 secondes
humidifier un moule à gâteau
Froisser une feuille de papier sulfurisé et la mettre au fond du moule
Verser la préparation
Enfourner à 170° pendant 35 à 50 minutes selon les fours
----->>>c'est parti pour la cuisson à 170° et durant 35 minutes
Enfin, mettre au frigo pendant 1 à 2h.
----->>>c'est parti pour le frigo pendant 60 minutes
Régalez vous !
```


Variables et Entrées/Sorties

Variables

Entrées/Sorties

La notion de **variable** peut être représentée sous forme de boîte accessible par son nom, dans laquelle on peut stocker la valeur d'une donnée qui dépend du type de donnée

yaourt



maïzena



"Bob"



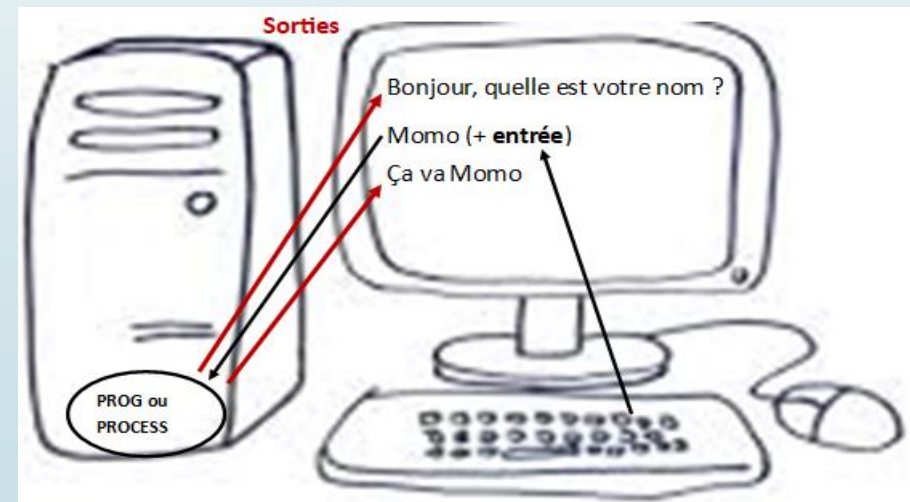
true



35



La notion d'**entrées** dans un programme indique ce qui vient de l'extérieur (saisie clavier) vers notre programme et inversement pour les **sorties** (affiche à l'écran)



Les variables varient dans le temps



9

Java 8
[known limitations](#)

```
1 public class YourClassNameHere {
2     public static void main(String[] args) {
3         int value = 10;
4         System.out.println( ++value ); //incrémente value = value + 1 et l'affiche -> 11
5         System.out.println("value = " + value);
6
7         System.out.println( --value ); //décrémente value = value - 1 et l'affiche -> 10
8         System.out.println("value = " + value);
9
10        System.out.println( value++ ); //affiche puis incrémente -> 10
11        System.out.println("value = " + value); // -> 11
12
13        System.out.println( value-- ); //affiche puis décrémente -> 11
14        System.out.println("value = " + value); // -> 10
15
16    }
17 }
```

[Edit this code](#)

→ line that just executed

→ next line to execute

<< First < Prev Next > Last >>

Step 12 of 15

Visualized with pythontutor.com

Print output (drag lower right corner to resize)

```
11
value = 11
10
value = 10
10
value = 11
```

Frames

Objects

main:13

value 11

Bien choisir ses variables et structures de données



C'est quoi une boucle ?



11

Exemple d'action unique que vous avez besoin de répéter plusieurs fois ? Faire une crêpe pardi !

En effet, si vous avez encore de la pâte à crêpes, vous pouvez répéter l'action de faire une crêpe.

Nous avons donc commencer par vérifier si une condition est réunie avant de passer à l'action.

Nous répétons cette action tant que la condition est vraie, sans quoi la boucle s'arrête.

Le choix de la condition est donc déterminant, il faut la formuler simplement :

« tant la condition permettant de répéter l'action est vraie, alors que je continue » « tant qu'il y a de la pâte à crêpe... » « tant que l'utilisateur veut jouer... » « tant que le nombre inconnu n'a pas été trouvé... »

```
public static void main(String [] args){
    Scanner scan = new Scanner(System.in);
    System.out.println("Bonjour souhaitez vous jouer à mon jeu O/N?");
    String str = scan.next();
    while(str.equalsIgnoreCase("oui")){
        int nbAleatoire = (int)(Math.random()*100) + 1;
        System.out.println(nbAleatoire);
        int counter = 1, val = 0;
        System.out.println("saisissez une valeur comprise entre 1 et 100");
        while(val != nbAleatoire){
            val = scan.nextInt();
            if( val > nbAleatoire) System.out.println("saisissez une valeur plus petite");
            else if( val < nbAleatoire) System.out.println("saisissez une valeur plus grande");
            else System.out.printf("vous avez trouvé en %d coups \n", counter);
            counter++;
        }
        System.out.println("Voulez vous rejouer ?");
        str = scan.next();
    }
    System.out.println("au revoir");
    scan.close();
}
```

Imaginer le code sans boucle ?

Attention au boucle infini voulu ou pas d'où la nécessité de débbugger

```
public static void main(String[] args) {
    for(int i = 0; i < 5 ; i++) System.out.print(i + " ");
    System.out.println();
    //-----
    int j = 0;
    for(;j<5;) System.out.print(j++ + " ");
    System.out.println();
    //-----
    int k = 0;
    while(k < 5) System.out.print(k++ + " ");
}
```

0	1	2	3	4
0	1	2	3	4
0	1	2	3	4

Quelle est la différence entre While et For ?

La boucle For est souvent utilisé quand on connaît exactement le nombre d'itération à l'avance alors que dans la boucle While, le nombre d'itération dépend plutôt d'une condition

Programme complet avec Automatisation du process

Variables

Entrées/Sorties

Tests

Boucles

```

public class RecetteGateauVersion2 {
    public static final int MIN_YAOURT = 400;
    public static final int MIN_OEUF = 4;
    public static final int MIN_MAIZENA = 40;

    public static void main(String[] args) throws InterruptedException {
        /*-----Programme de réalisation d'un gâteau-----*/
        System.out.println("Bienvenu dans notre programme qui réalise des gâteaux");

        /*-----liste des ingrédients-----*/
        System.out.println("nous allons vérifier si vous disposez bien des ingrédients en quantité suffisante ");
        Scanner scan = new Scanner(System.in);
        System.out.println("saisissez la quantité de yaourt dont vous disposez ?");
        int yaourt = scan.nextInt();
        if (yaourt >= MIN_YAOURT) { // avec 1000g on peut faire 2 gateaux / avec 200g on peut en faire aucun
            System.out.println("saisissez le nombre d'oeufs dont vous disposez ?");
            int oeuf = scan.nextInt();
            if (oeuf >= MIN_OEUF) { // avec 9oeufs on peut faire 2 gateaux / avec 1 oeuf, on peut en faire aucun
                System.out.println("saisissez la quantité de Maizena dont vous disposez ?");
                int maizena = scan.nextInt();
                if (maizena >= MIN_MAIZENA) {
                    fabriquonsDesGateaux(yaourt, oeuf, maizena);
                } else System.out.println("vous n'avez pas suffisamment de Maizena");
            } else System.out.println("vous n'avez pas suffisamment d'oeufs");
        } else System.out.println("vous n'avez pas suffisamment de yaourt");
    }

    private static void fabriquonsDesGateaux(int yaourt, int oeuf, int maizena) throws InterruptedException {
        //Combien de gateaux pouvons fabriquer ?
        int nbGateauYaourt = yaourt / MIN_YAOURT;
        int nbGateauOeuf = oeuf / MIN_OEUF;
        int nbGateauMaizena = maizena / MIN_MAIZENA;

        int nbGateau = 0;
        if(nbGateauYaourt <= nbGateauOeuf) nbGateau = nbGateauYaourt;
        else nbGateau = nbGateauOeuf;
        if(nbGateau >= nbGateauMaizena) nbGateau = nbGateauMaizena;

        System.out.println("le nombre total de gateau que nous pouvons réaliser : " + nbGateau);
        for(int i=0 ; i<nbGateau ; i++) {
            System.out.println("Réalisation du gâteau numéro " + (i+1));
            System.out.println("mélanger les yaourts avec les oeufs pendant 10 secondes");
            Thread.sleep(10000); //moyen pour donner l'impression qu'on mélange pendant 10 secondes

            System.out.println("humidifier un moule à gâteau");
            Thread.sleep(5000); //opération prend 5 secondes

            System.out.println("Froisser une feuille de papier sulfurisé et la mettre au fond du moule");
        }
    }
}

```

C'est quoi une méthode ?

Dans l'exemple précédent, nous avons utilisé la notion de méthode permettant d'apporter une meilleure lisibilité en déportant une partie de l'algorithme ailleurs dans le programme. Du coup, nous avons centralisé une partie de l'appli dans une méthode qui a vocation à réaliser une ou plusieurs instructions avec la possibilité de renvoyer (fonction) ou pas (procédure) une donnée (ou un ensemble). Noter au passage qu'on aurait pu s'en passer mais il aurait fallu écrire partout où c'est nécessaire cette suite d'instructions, on parle aussi ici de factorisation de code.

```
public class Score {  
    public static void main(String[] args) {  
        double [] scores = { 10,13,17,20,8.5};  
  
        displayScores(scores);  
        System.out.println();  
  
        System.out.println("Max = " + maxScores(scores));  
        System.out.println("Min = " + minScores(scores));  
        System.out.println("Moy = " + averageScores(scores));  
    }  
  
    public static void displayScores(double [] scores) {  
        for(double n : scores) System.out.print(n + " ");  
    }  
  
    public static double maxScores(double [] scores) {  
        double max = scores[0];  
        for(int i = 1 ; i < scores.length ; i++) {  
            if(max < scores[i])  
                max = scores[i];  
        }  
        return max;  
    }  
}
```

Algorithmique : méthode pour réaliser des algos (1)

- Un **algorithme** sert à répondre à un **besoin** : « fabriquer un gâteau »
- Il est **constitué** d'un **début** et d'une **fin**
- Il peut contenir : **variables**, **entrées/sorties**, **tests** et **boucles**
- La **qualité** d'un algorithme dépend de notre capacité à gérer correctement le(s) **cas général** et les **cas particuliers** en **anticipant** sur toutes les situations/chemins possible
 - Tous les ingrédients sont réunis en quantité suffisante, on peut réaliser un ou plusieurs gâteaux
 - Un ou plusieurs ingrédients manque aussi, il faut indiquer à l'utilisateur la raison pour laquelle, on ne pourra pas le satisfaire.
- **La phase de test** permet de s'assurer que notre algorithme fonctionne comme prévue et de le valider par un minimum de **couverture de test** visant à passer par tous les chemins : **jeux d'essai, test unitaire**
- La **simplicité** engendre la **lisibilité** de l'algorithme par des collègues aussi il faut utiliser des **noms explicites**, une **indentation** clair, des **commentaires** précis et suffisant, des **méthodes** permettant de factoriser le code (éviter les répétitions inutiles)
- *NB : Lorsque vous sous traitez tout ou partie de votre programme à une ressource externe(ex : Scanner), elle ne doit avoir aucun secret pour vous, ça passe par une lecture précise de la documentation ou par une manipulation poussée qui prend du temps.*

Cas général & cas particuliers

Programmes	Cas général (Situation attendue la + courante)	Cas particuliers (situation attendue - courante)
Ex3 ou gestion de nombre positif/négatif & Pair/impair	Cas 1 : $X > 0$ Cas 2 : $x < 0$ Cas 3 : X pair...	$X == 0$ Saisie invalide
Tp Game	$X > \text{nbAléatoire}$ $X < \text{nbAléatoire}$ $X == \text{nbAléatoire}$ Rejouer = 0	Rejouer = N Saisie invalide
Gestion d'une Horloge	Seconde++ Si sec == 60 alors sec = 0 & minute++ Si min == 60 alors min = 0 & heure ++	Si heure == 23 && Minute == 59 && Seconde == 59 alors h=0...
Tp Resto	???	???

Pour les plus motivés : [Mise en œuvre d'un Algo + costaud pas à pas](#)

Algorithmique : méthode pour réaliser des algos (2)



"Tout le monde dans ce pays devrait apprendre à programmer un ordinateur, parce que cela vous apprend à réfléchir" — Steve Jobs

Tous les jours nous rencontrons des problématiques diverses perso ou prof, et nous essayons d'y répondre en testant plusieurs options, perdant beaucoup de temps aussi pour être efficace il faut : méthodes + gestion des priorités :

- *"La plus grande erreur que j'observe chez les jeunes programmeurs est de se focaliser sur la syntaxe plutôt que d'apprendre comment résoudre des problèmes" — V.Anton Spraul*

- 1/ Analyser & comprendre les besoins ou la problématique (un minimum)

- *"Si vous n'arrivez pas à expliquer quelque chose avec des termes simples, c'est que vous ne le comprenez pas" — Richard Feynman*

- *"La plus grande erreur que j'observe chez les développeurs juniors est de foncer tête baissée sans prendre le temps de la réflexion, le codage pur est pourtant qu'une petite partie du métier" — M.E*

- 2/ Réfléchir et Planifier la solution selon des étapes bien distinctes

- Commencer par écrire une road map cohérente sous forme de commentaires : Ecrire en français les étapes puis coder chaque étape

- 3/ Diviser pour régner

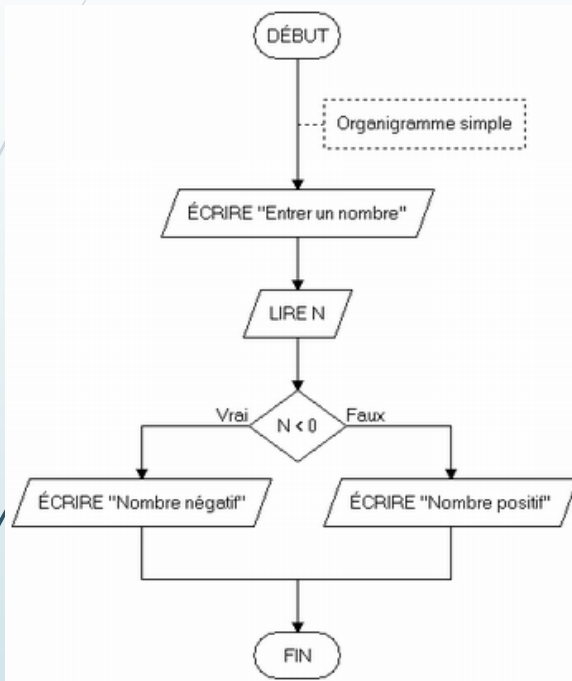
- Découper un problème complexe en petit pb + simple, programmation modulaire...

- 4/ Syndrome de la page blanche

- Mieux vaut faire quitte à défaire que de rester à tourner en rond puis revenir à l'étape 2

Outil visuel d'aide à la réalisation d'algo

Algorithme ou organigramme

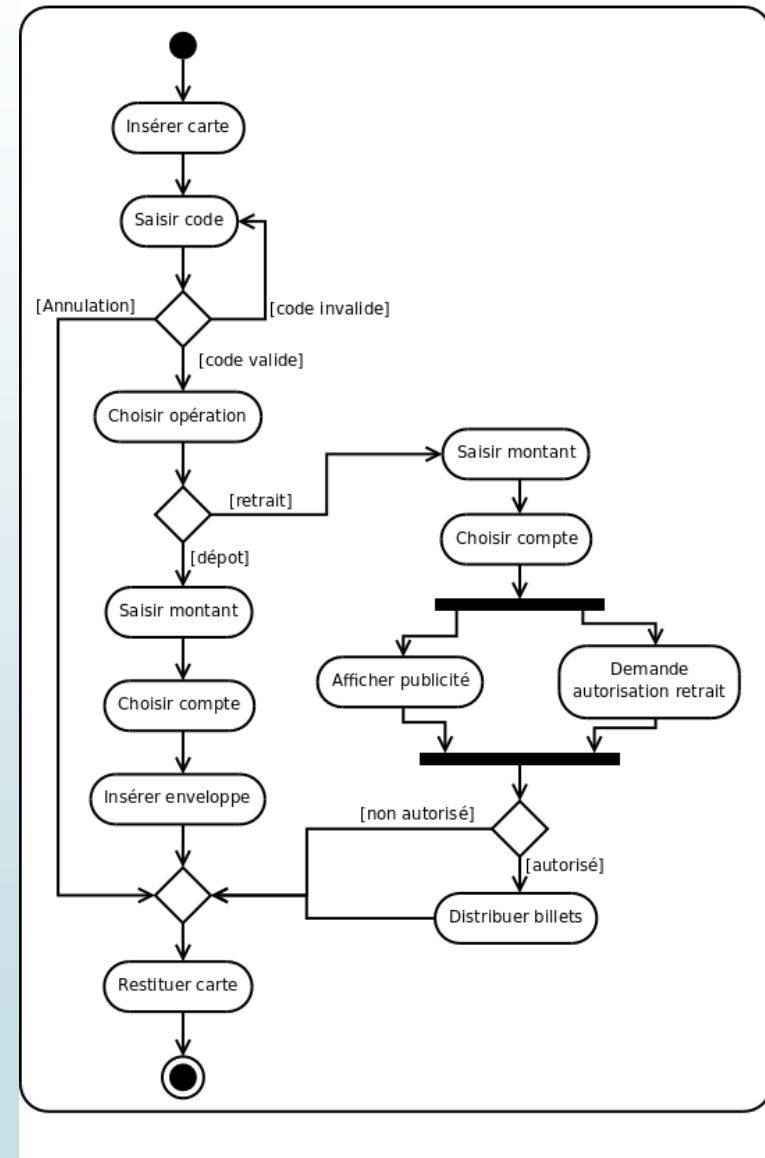


```
Debut
|
| Lire N
| Si N > 0 alors retourner "nombre positif"
| Sinon si N < 0 alors retourner "nombre négatif"
Fin

*****

Debut
|
| Lire N
| Si N >= 0 alors retourner "nombre positif"
| Sinon retourner "nombre négatif"
Fin
```

Diagramme d'activité (UML)



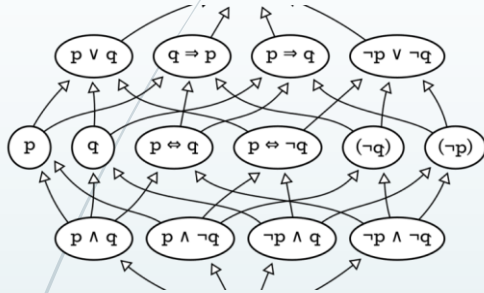
Déboguer : comment résoudre les imprévus ?

18

- *"L'art du débogage consiste à comprendre exactement ce que vous dites à votre programme plutôt que ce que vous pensez lui avoir dit de faire" — Andrew Singer*
- Avant d'utiliser les outils de débogage qui aide à résoudre beaucoup de pb :
 - prendre du recul, de la hauteur, un autre point de vue au lieu de rester la tête dans le guidon.
 - Revenir donc à l'essentiel, aux fonctionnalités principales avant de rezoomer sur les secondaires.
 - Reprendre parfois tout depuis le début peut prendre du temps mais peut être indispensable pour comprendre que l'erreur remonte au tout début.
- Avec un déboguer, le mode pas à pas et les points d'arrêts est une manière efficace mais il ne faut pas se reposer entièrement sur celui-ci, ça empêche d'analyser un programme par soi même.
- Toujours privilégier l'entretien de votre muscle cérébral par la lecture et compréhension de codes
- S'agissant d'un point spécifique, stackoverflow par ex peut vous aider à condition d'être précis dans votre recherche, sans quoi vous risquez de vous noyer dans l'océan d'information, aussi votre recherche doit résulter d'une véritable réflexion en amont :
 - Quelle est le périmètre technique de ce que je recherche ? Sql ou Orm/Jpa/Hibernate/Spring Data
 - Formuler clairement votre recherche : qu'est-ce que je recherche ? Pour répondre à quel besoin ?
 - Il faut comprendre comment fonctionne les moteurs de recherche afin d'utiliser les bons mots clés
 - Vous devez suffisamment connaître votre périmètre technique sans avoir besoin de le maîtriser !
 - Enfin aujourd'hui, dans le cadre des apprentissages, **ChatGpt** peut être un frein mais pour vous aider à trouver de l'info pertinente en un temps records, l'outil est top, enfin ça dépend de comment vous l'exploitez !

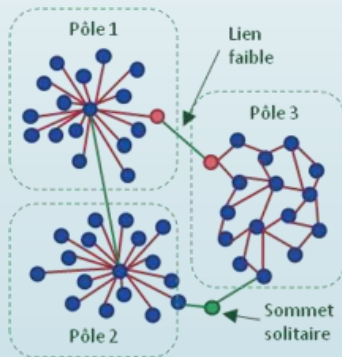
➡ *"Juste au moment où vous pensez avoir franchi un obstacle, un nouveau apparaît. Mais c'est ce qui rend la vie intéressante. [...] La vie est un processus consistant à trouver des solutions aux obstacles, les uns après les autres. A chaque fois, vous apprendrez quelque chose. A chaque fois, vous développerez votre force, votre sagesse et votre perspective. A chaque fois, les difficultés tomberont. Jusqu'à ce que tout ce qu'il reste se résume à vous : la meilleure version de vous." — Ryan Holiday (The Obstacle is the Way)*

Pour aller + loin : Maths !



L'algèbre de Boole, ou calcul booléen :

C'est la partie des mathématiques qui s'intéresse à une approche algébrique de la logique, vue en termes de variables, d'opérateurs et de fonctions sur les variables logiques



La théorie des graphes :

C'est la discipline mathématique et informatique qui étudie les graphes, lesquels sont des modèles abstraits de dessins de réseaux reliant des objets.

QQ critères d'une application de qualité

Efficacité : un maximum de résultat pour un minimum d'effort

Fiabilité : tests unitaires et d'intégration

Robustesse : capacité à résister aux erreurs

Maintenabilité : fermée aux modifications, ouverte aux évolutions

Versionning : notamment lorsqu'on travaille en équipe

Portabilité : gestion de changement de bibliothèques ou base de donnée

Performance : la rapidité d'exécution d'une méthode ou d'une requête

Process : Le respect des règles garanti la cohésion d'une équipe, un wagon dans le train

Répéter pour maîtriser

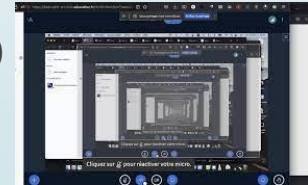


- Le secret qui permet de viser l'excellence dans chaque métier reste la répétition des gestes techniques
- En effet, qu'est-ce qui fait qu'un hôpital acquière une notoriété dans telle ou telle chirurgie ? Telle opération y est réalisée de nombreuses fois/an
- De même, la répétition dans l'analyse des besoins exprimés ou problématique à solutionner et la réponse ou solution apportée, doit être un processus scrupuleusement suivi et suffisamment reproduit afin d'acquérir les bons réflexes
- Ceci dit, il y a des domaines où les algo sont + ou – complexes et volumineux nécessitant une double expertise en amont en math/physique par ex :
 - Scientifique (traitement du signal...)
 - Gestion (web...)
 - Industriel (embarqué...)
 - Aéronautique (Plm...)

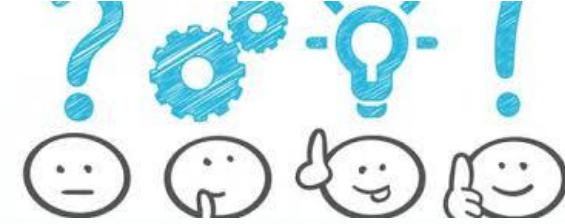
Comment optimiser un algorithme et améliorer la qualité du code ?

22

- Il faut de l'expérience, ce n'est pas quelque chose qui doit être visé dans un 1^{er} temps puisque ça risque de rendre la tâche plus compliquée aussi concentrez vous surtout sur :
 - Est-ce que ça fonctionne ? Est-ce que ça répond aux besoins ?
 - Est-ce que ça gère tous les cas ? Est-ce simple à comprendre ?
- Une fois le 1^{er} cap passé, voici des pistes :
 - Utiliser les boucles uniquement quand c'est indispensable
 - Factoriser quand c'est possible en réduisant les tests, ça réduit le tps de calcul, utilisant les méthodes...
 - En minimisant l'occupation en mémoire (embarqué , éviter la récursivité...)
- Il existe des outils pour nous imposer des règles d'écriture de code qui facilite grandement le déploiement d'une application puisque les pbs ont été détectés tôt, ex : sonar...



Prise de sens ou déclic



- Le métier a beaucoup évolué depuis 20 ans, la pile technologique constituant votre mallette est devenu conséquente, aussi il faut être prêt à apprendre à apprendre en permanence.
- L'autre difficulté vient des concepts abstraits qu'on ne peut se représenter qu'à l'aide de dessins de vulgarisation, d'outils pédagogiques + ou - pertinents, d'imagination, de l'expérience ou du rythme d'apprentissage.
- Raisons pour lesquelles, **la prise de sens** peut arriver à des moments différents selon chacun(e) et c'est tout à fait normal, ce sont les fameux déclics !



Paradigme de programmation

- Un paradigme de programmation fournit (et détermine) la vue qu'a le développeur de l'exécution de son programme. Par exemple, en programmation orientée objet, les développeurs peuvent considérer le programme comme une collection d'objets en interaction, tandis qu'en programmation fonctionnelle un programme peut être vu comme une suite d'évaluations de fonctions sans états.
- Le problème vient lorsqu'on connaît qu'un seul paradigme, on va répondre aux problèmes qu'en fonction de celui-ci. Ce qui peut se traduire par exemple par l'utilisation d'une porche pour acheter une baguette au bout de la rue.
- En réalité, les langages évolués comme Python ou Java nous permettent d'utiliser plusieurs paradigmes. Nous allons utiliser le paradigme procédurale dans un premier temps puis nous basculerons sur celui de la Programmation orientée objet avec JAVA

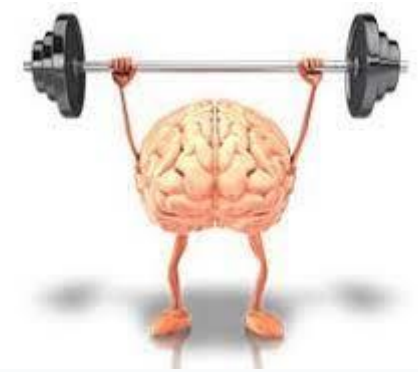
C'est quoi le métier de développeur ?

25

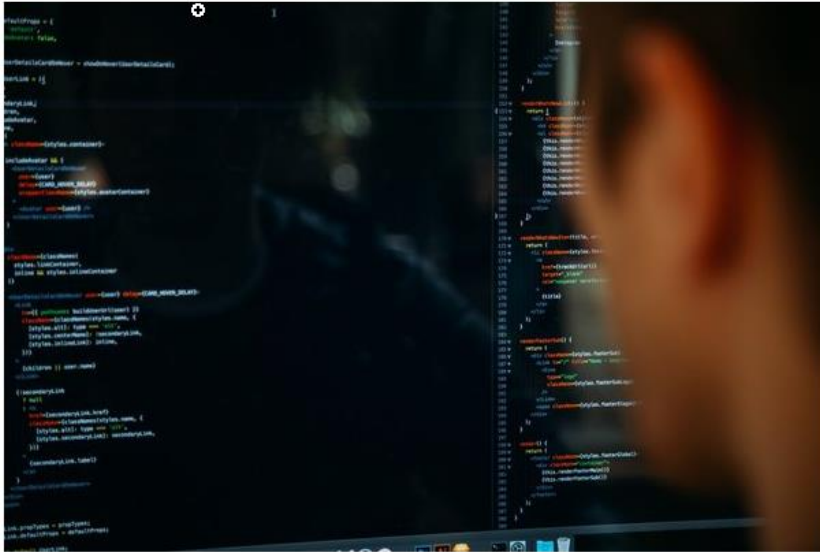


- Tout le monde peut coder au même titre que tout le monde peut faire des maths à condition de bien lire les leçons et les consignes puis de faire de nombreux exercices.
 - Tous les développeurs ont commencé au niveau 0 un jour, n'est-ce pas ?
- En revanche, tous n'avancent pas au même rythme selon leur parcours et leur motivation
- Softs Skills du dev :
 - **Esprit Logique/Méthodique/Analytique, Bon sens/Intuition, Aimer résoudre des problématiques complexes, échanger et confronter ses idées, Sens de la communication et du service, Capacité de travailler en équipe, respect des consignes/normes, curieux, Agile/Adaptable, innovant, Aime apprendre de nouvelles choses, Esprit synthétique, prise de décision, créativité, sens de l'organisation, Esprit logique de déduction, Savoir quoi chercher comment et où, Motivé, aime les challenges, persévérant, innovant, technique, ingénieux, autonome, anglophone**
- Comment faire pour les développer ?
 - Il faut se remettre au sport ... Intellectuel
 - La moindre occasion pour coder doit être saisie
 - Il y aura une phase de décrassage, de résistances, de doutes et de détermination
 - Réaliser toutes les demandes dans la mesure du possible
 - Apprendre à apprendre
 - Privilégier les activités saines pour le cerveau
 - Notre capacité à se concentrer étant limité, parfois il faut savoir switcher pour éviter les migraines

Durée de concentration maximale Par heure ? Par jour ?



Nous avons effectué une étude à la NASA il y a longtemps et je pense que les résultats sont toujours valables.



- Sélectionnez deux équipes de développeurs, en faisant de votre mieux pour qu'ils soient de même niveau.
- Attribuez-leur le même problème.
- Donnez à l'une des équipes une échéance irréaliste et à l'autre une échéance réaliste.

L'équipe au délai irréaliste a travaillé pendant plus longtemps sur le projet (en terme d'heures) - ils ont fini le projet quasiment au même moment que l'autre équipe, cependant le résultat produit contenait bien plus de bugs et était plus difficile à maintenir.

Le résultat de cette étude n'a pas été suffisamment supporté par la hiérarchie pour justifier une publication, mais on en a conclu que: sur la durée d'un projet, **il est préférable que les personnes travaillent moins d'heures.**

60 % des devs
consacrent max 4h de
devs par jour

Voir aussi la technique [Pomodoro](#)

Pour un cerveau en bonne santé



- Alimentation saine (huile d'olive, poissons, fruits, légumes...) **vs** Sucre (drogues)
- Sport (cœur, cerveau, mémoire) **vs** apathie
- Méditation/relaxation/silence **vs** stress
- Qualité du sommeil **vs** écrans tard le soir
- Jeux de réflexions **&** jeux vidéos
- Culture (lecture, dessin, peinture, théâtre, cinéma...) **vs** télévision uniquement
- Multiplier les échanges sociaux et les débats **vs** les mêmes habitudes
- Jardiner, Cuisiner, Couturer, bricoler...
- Jouer d'un instrument
- Apprendre une langue étrangère
- Apprendre de nouvelles choses chaque jour
- Voyager même à côté (terre, montagne, mer et océan...)
- Avoir un projet professionnel **vs** aucun projet de vie
- **Motivation** : *Tout ce que vous donnez à votre corps de bon, reviendra un jour !*

Un dernier mot

28

- Souvent vous entendrez, les informaticiens sont des fainéants ou il ne faut **pas réinventer la roue** ... C'est vrai quand il s'agira de répondre à des besoins de + en + complexe, il faut voir s'il n'existe pas un algo qui fait le job quitte à l'adapter un peu.
- Cela n'est pas valable dans votre contexte d'apprentissage, vous devez réaliser tous les algos afin de **muscler votre raisonnement informatique**.
- Ne jamais hésiter à **ouvrir le capot** pour regarder ce qu'il y a dedans afin de s'appropriier les choses, **ne pas avoir peur de se tromper** c'est comme ça aussi qu'on apprend.
- Dans le contexte airbus, ce n'est pas toujours simple mais trouver un moyen de **faire un labo de test** afin de manipuler les choses sans impacter les applis.
- Le métier de développeur est fantastique avec des challenges toujours plus ambitieux, seulement parfois, il y a des taches répétitives simples qu'il faut faire un peu comme le maçon qui monte un mur...

Ressources

29

- <https://algo.developpez.com/cours/>
- <http://cours.pise.info/algo/introduction.htm>
- <https://medium.com/@rhunold/comment-penser-comme-un-programmateur-le%C3%A7on-pour-pour-r%C3%A9soudre-des-probl%C3%A8mes-1f374a4b399f>
- <https://pythontutor.com/java.html#mode=edit>
- <https://www.france-ioi.org/index.html>
- [50 conseils pratiques](#)